Introduction to Computer Science

Unit 3:

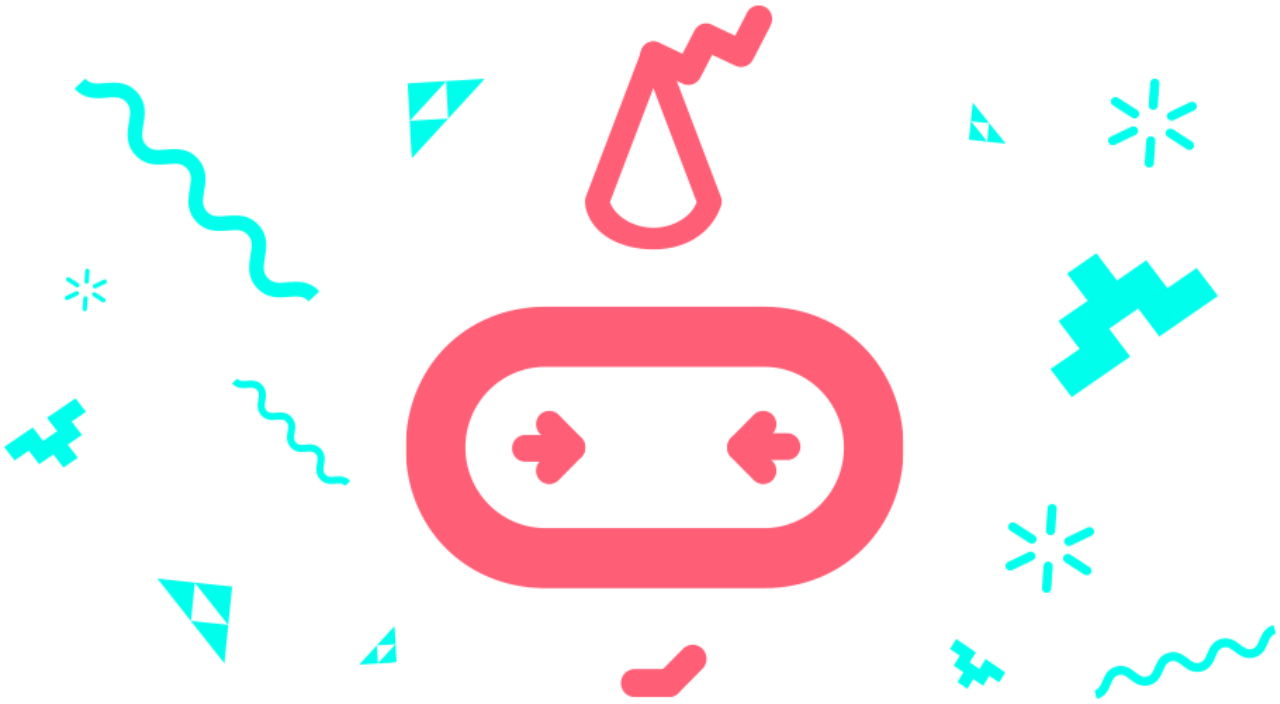
Variables

Student workbook

makecode.microbit.org

Microsoft

Table of Contents



Overview

Unit summary

This unit introduces the use of variables to store information. You will practice giving variables unique and meaningful names and use basic mathematical operations for adding, subtracting, multiplying, and dividing variable values. In the unplugged activity, you will learn to differentiate between constants and variables by tracking scores as you play a simple game of "Rock, Paper, Scissors". Then you will code a program for your micro:bit that keeps and displays the score of the game, using the programmable buttons for input and the LED screen for output. In the final project, you will code your own unique program using variables and design and build an object that uses the micro:bit to track score, count steps, turns, or something else.

Learning goals

During this unit, you will:

- Understand what variables are and why and when to use them in a program.
- Learn how to create a variable, set the variable to an initial value, and change the value of the variable within a micro:bit program.
- Learn how to create meaningful and understandable variable names.
- Understand that a variable holds one value at a time.
- Understand that when you update or change the value held by a variable, the new value replaces the previous value.
- Learn how to use the basic mathematical blocks for adding, subtracting, multiplying, and dividing variable values.
- Apply the above knowledge and skills to create a unique program that uses variables as an integral part of the project.

Lesson A: Variables in daily life

Overview: Variables

From our previous lesson, you learned that computer programs process information. Some of the information that is input, stored, and used in a computer program has a value that is constant, meaning it does not change throughout the course of the program. An example of a constant in math is 'pi' because 'pi' has one value that never changes (it is approximately 3.14). Other pieces of information have values that vary or change during the running of a program. Programmers create variables to hold the value of information that may change. In a game program, a variable may be created to hold the player's current score, since that value would change (hopefully!) during the course of the game.

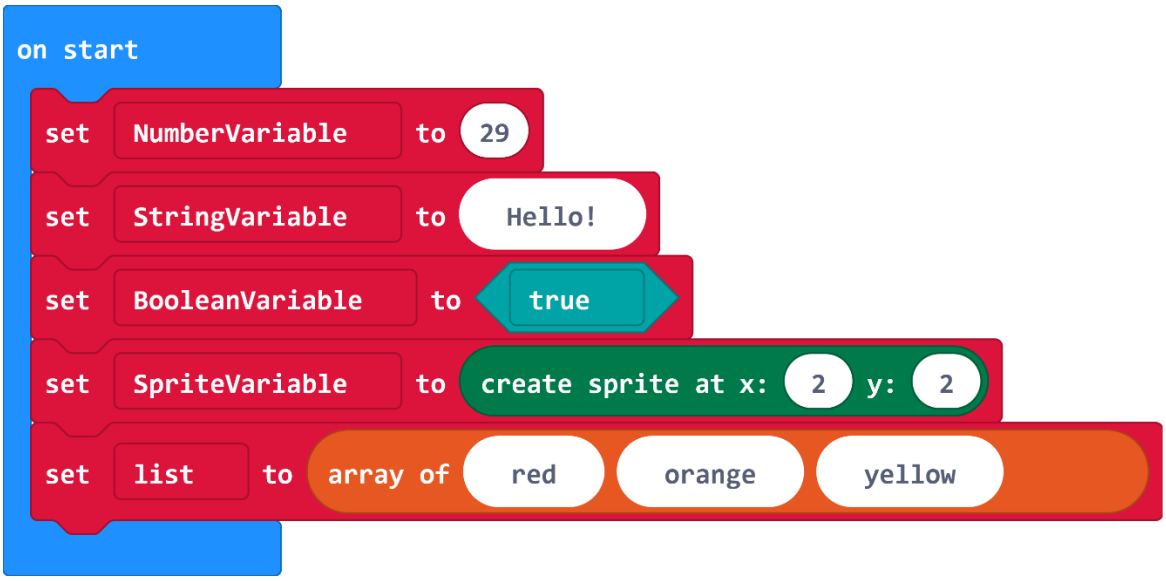
Constants and variables can be numbers and/or text.

Examples

In one school day…

- *Constants*: The day of the week, the year, student's name, the school's address, the student's birthday
- *Variables*: The temperature/weather, the current time, the current class, whether members of the class are standing or sitting…

Variables hold a specific type of information. The micro:bit's variables can keep track of **numbers, strings, Booleans**, **sprites**, and **arrays**. The first time you use a variable, its type is assigned to match whatever it is holding. From that point forward, you can only change the value of that variable to another value of that same type.



- A **number variable** could hold numerical data such as the year, the temperature, or the degree of acceleration.
- A **string variable** holds a string of alphanumeric characters such as a person's name, a password, or the day of the week.
- A **Boolean** variable has only two values: true or false. For example, you might have certain things that happen only when the variable called GameOver is false.
- A **sprite** is a special variable that represents a single dot on the screen and holds two separate values for the row and column the dot is currently in.
- An **array** is another special type of variable that holds a list of multiple items.

Lesson B: Make a game scorekeeper

Coding activity: Scorekeeper

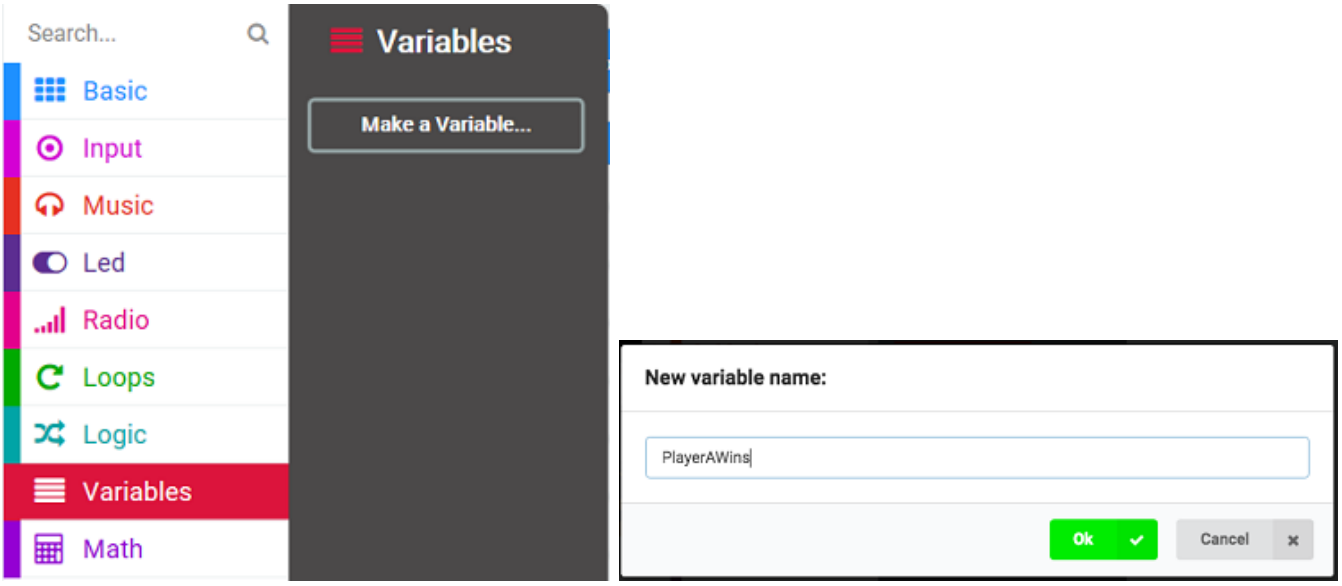
Overview

You will be creating a program that will act as a scorekeeper for your next "Rock, Paper, Scissors" game. You will need to create variables for the parts of scorekeeping that change over the course of a gaming session.

What are those variables?

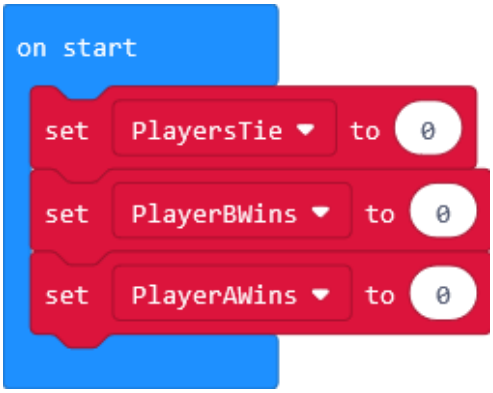
- The number of times the first player wins
- The number of times the second player wins
- The number of times the players tie

1. In MakeCode, start a new project and name it something like: **Scorekeeper**. From the Variables menu, make and name these three variables: **PlayerAWins**, **PlayerBWins**, **PlayersTie**.



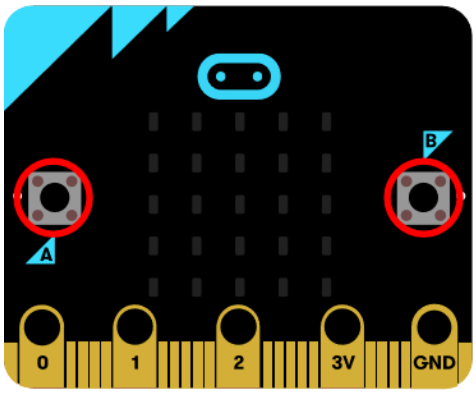
Initializing the variable value

2. It is important to give your variables an initial value. The initial value is the value the variable will hold each time the program starts. For our counter program, we will give each variable the value 0 (zero) at the start of the program.



Updating the variable value

In your program, you want to keep track of the number of times each player wins and the number of times they tie. You can use the buttons A and B on the micro:bit to do this.



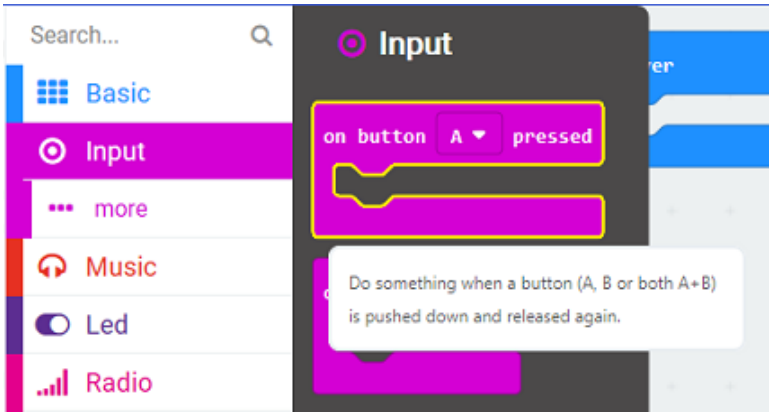
The pseudocode, written in words, is:

- Press button A to record a win for Player A
- Press button B to record a win for Player B
- Press both button A and button B together to record a tie

We already initialized these variables and now need to code to update the values at each round of the game.

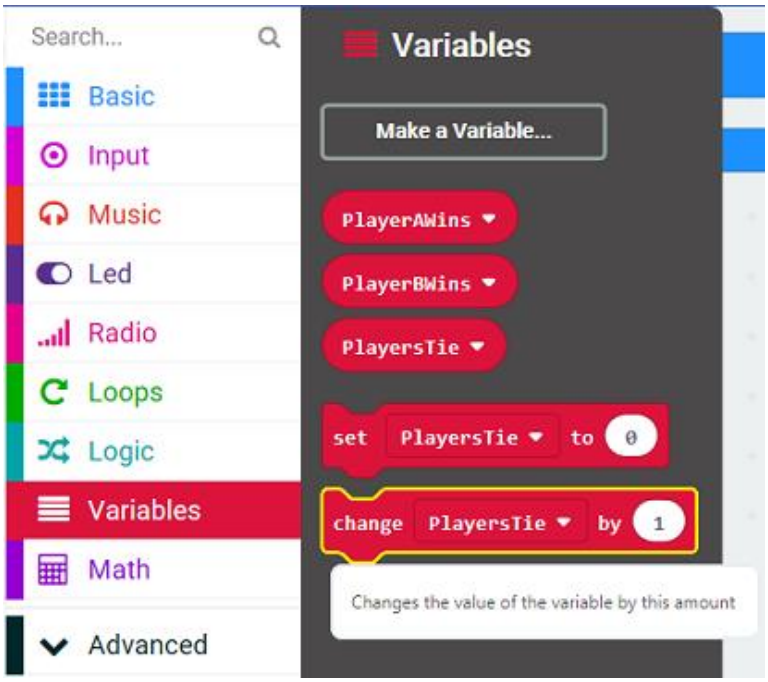
- Each time the scorekeeper presses button A to record a win for Player A, you want to add one to the current value of the variable PlayerAWins.
- Each time the scorekeeper presses button B, to record a win for Player B, you want to add one to the current value of the variable PlayerBWins.
- Each time the scorekeeper presses both button A and button B at the same time to record a tie, you want to add one to the current value of the variable PlayersTie.

3. From the Input menu, drag three of the '**on button A pressed**' blocks to your Programming Workspace.



Leave one block with '**A**'. Use the dropdown menu in the block to choose '**B**' for the second block and '**A+B**' for the third block.

4. From the Variables menu, drag three of the '**change variable by 1**' blocks to the coding Workspace.



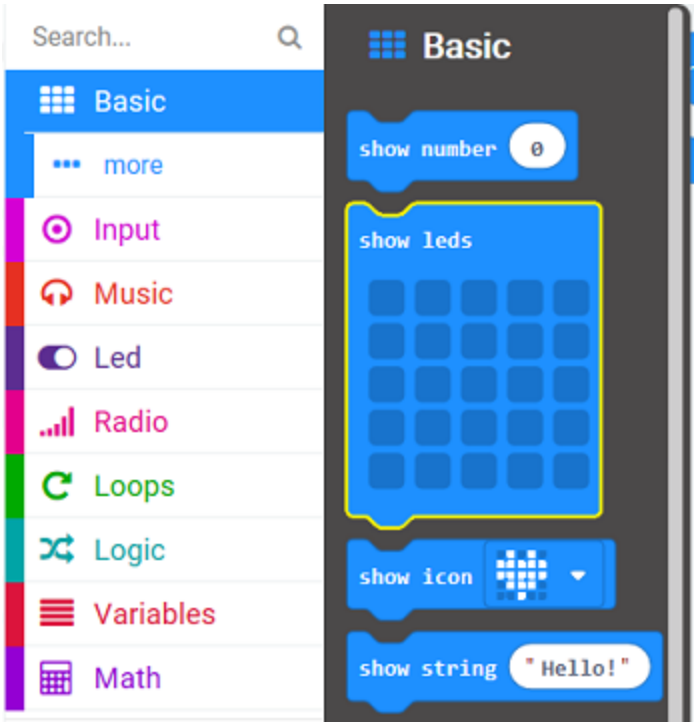
5. Place one change variable block into each of the '**on button pressed**' blocks.
6. Choose the appropriate variable from the pull-down menus in the '**change by 1**' blocks.

Adding user feedback

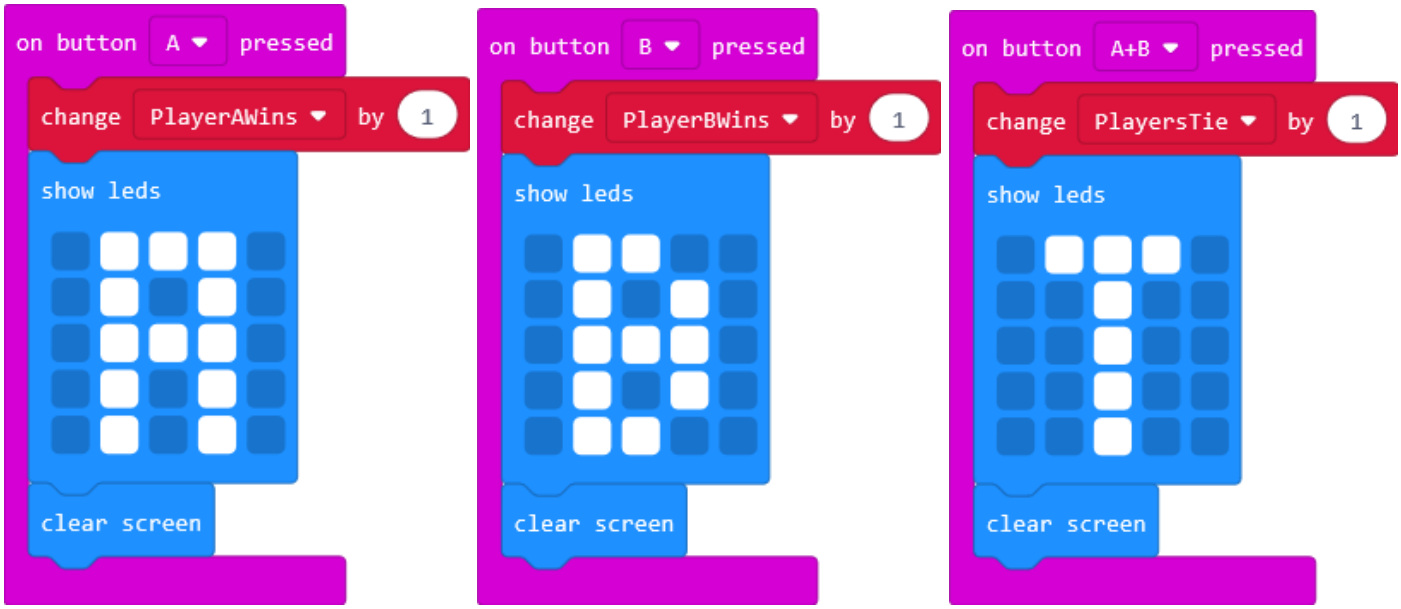
Whenever the scorekeeper presses button A, button B, or both buttons together, you will give the user visual feedback acknowledging that the user pressed a button. You can do this by coding your program to display:

- An 'A' each time the user presses button A to record a win for Player A
- A 'B' for each time the user presses button 'B' to record a win for Player B
- A 'T' for each time the user presses both button A and button B together to record a tie

7. We can display an 'A', 'B', or 'T' using either the '**show leds**' block or the '**show string**' block.



In this example, the '**show leds**' block is used.

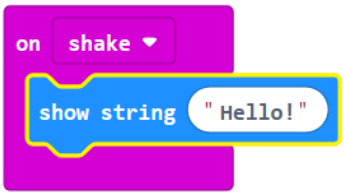


Notice that you added a **'clear screen'** block after showing 'A', 'B', or 'T'. What do you think would happen if you did not clear the screen? Try it.

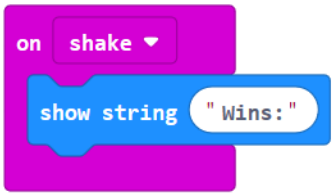
Showing the final values of the variables

To finish your program, you can add code that tells the micro:bit to display the final values of our variables. Since we have already used buttons A and B, we can use the 'on shake' event handler block to trigger this event. We can use the 'show string', 'show leds', 'pause', and 'show number' blocks to display these final values in a clear way.

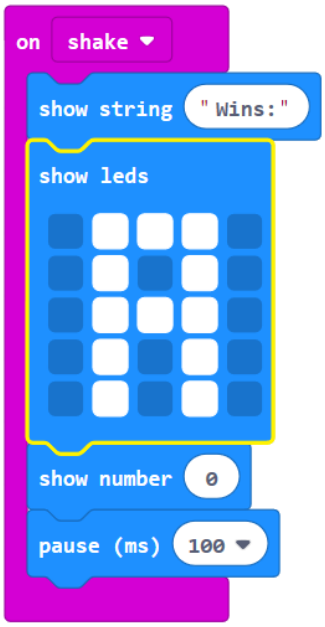
8. From the Input Toolbox drawer, drag an **'on shake'** block to the coding Workspace. Then, from the Basic Toolbox drawer, drag a **'show string'** block inside it.



Change the text of the string from "**Hello!**" to: "**Wins:**"



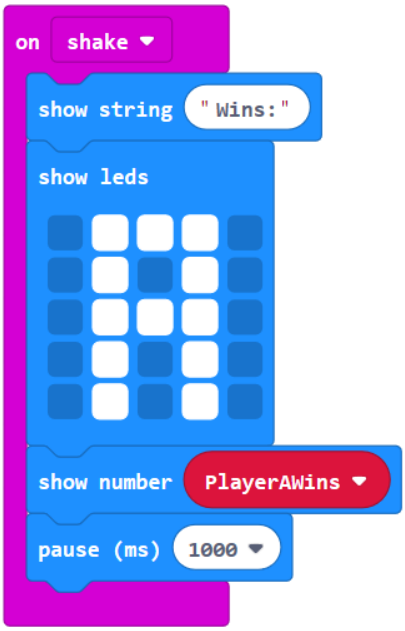
9. From the Basic Toolbox, drag a **'show leds'**, a **'show number'**, and a **'pause'** block to the coding Workspace and connect them under the **'show string'** block. Select the boxes in the **'show leds'** block to make the letter A.



10. When you shake the micro:bit, you want it to display 'Wins: ', then, display the letter A followed by the number of wins for Player A.

    a. From the Variables Toolbox, drag the '**PlayerAWins**' variable to replace the 0 in the '**show number**' block.

    b. To add an adequate pause before showing the number of wins for Player B, use the dropdown menu in the '**pause**' block and select **1 sec** (which is 1000 ms).



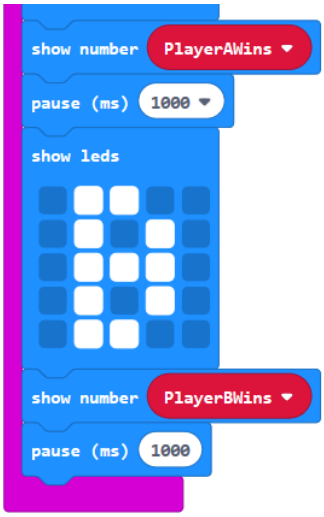
11. Now, you can follow the same steps to show the number of wins for Player B.

    a. From the Basic Toolbox, drag the '**show leds**', '**show number**', and '**pause**' blocks to the coding Workspace and add them under the '**pause (ms) 1000**' block.

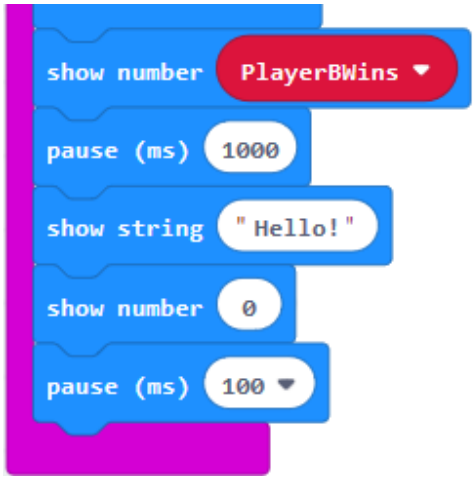
    b. Select the boxes in the '**show leds**' block to display the letter B.

    c. From the Variables Toolbox, drag the '**PlayerBWins**' variable to replace the **0** in the '**show number**' block.

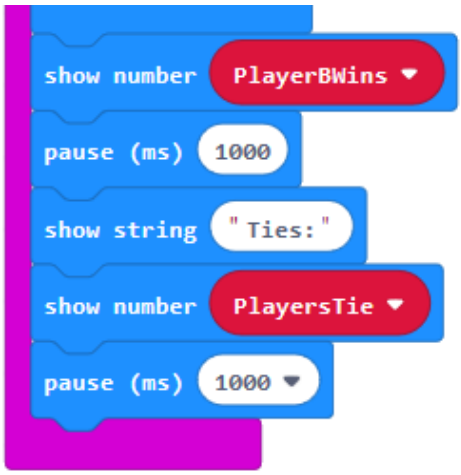
    d. In the '**pause**' block, use the dropdown menu, and select **1 second**.



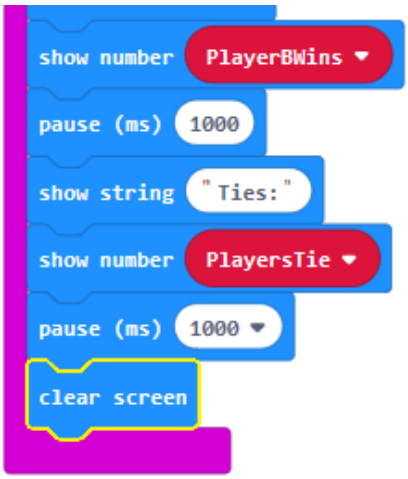
12. To show the number of ties, go to the Basic Toolbox, and drag the '**show string**', '**show number**', and '**pause**' blocks underneath the last '**pause (ms) 1000**' block—and still inside the '**on shake**' block.



13. In the '**show string**' block, change the "**Hello!**" to "**Ties:** " Then, from the Variables Toolbox, drag the '**PlayersTie**' variable block to replace the **0** in the '**show number**' block. And change the '**pause**' block to **1 second**.



14. There is one last block to add. Can you guess what it is? From the Basic Toolbox, drag a '**clear screen**' block underneath the final '**pause**' block.



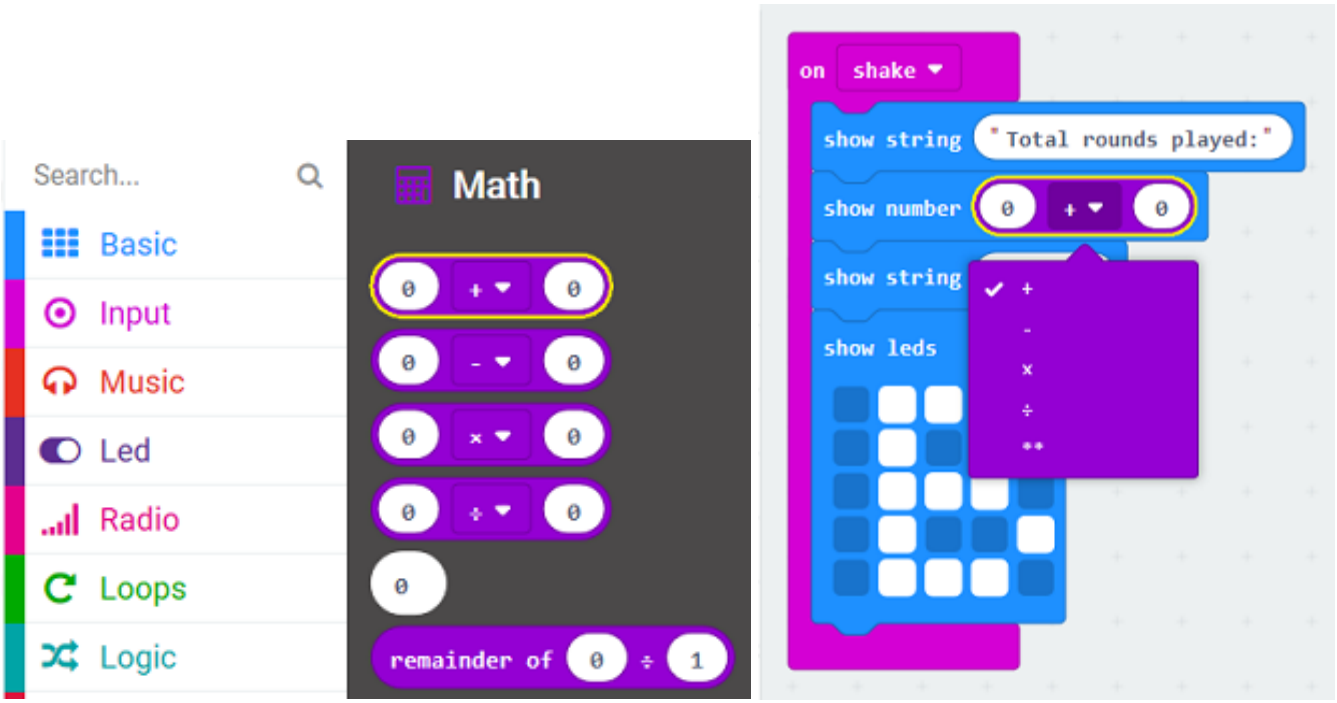
Adding on with mathematical operations

There is more you can do with the input you received using this program. You can use mathematical operations on your variables.

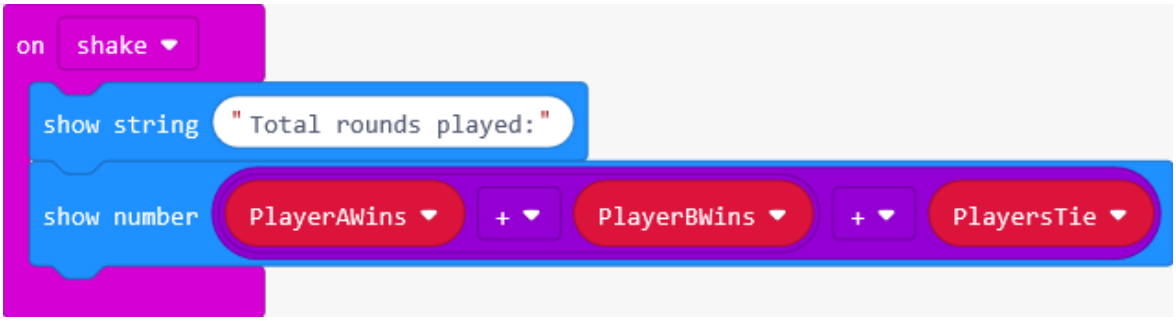
Example: Perhaps you'd like to track and show the player the total number of rounds that were played. To do this, you can add the values stored in the variables you created to keep track of how many times each player won and how many times they tied.

15. In order to do this, you can add the code to your program under the 'on shake' event handler.
    a. First, display a string to show the player that the following sum represents the total number of rounds played.
    b. Our program will add the values stored in the variables PlayerAWins, PlayerBWins, and PlayersTie, and then display the sum of this mathematical operation.
    c. The blocks for the mathematical operations adding, subtracting, multiplying, and dividing are listed in the Math section of the Toolbox.

Even though there are four blocks shown for these four operations, you can access any of the four operations from any of the four blocks, and you can also access the exponent operation from these blocks.



16. Replace the default values of 0 with the names of the variables we want to add together. Notice that because you are adding three variables together, you need a second math block. First, add the values for PlayerAWins and PlayerBWins, then add PlayersTie.



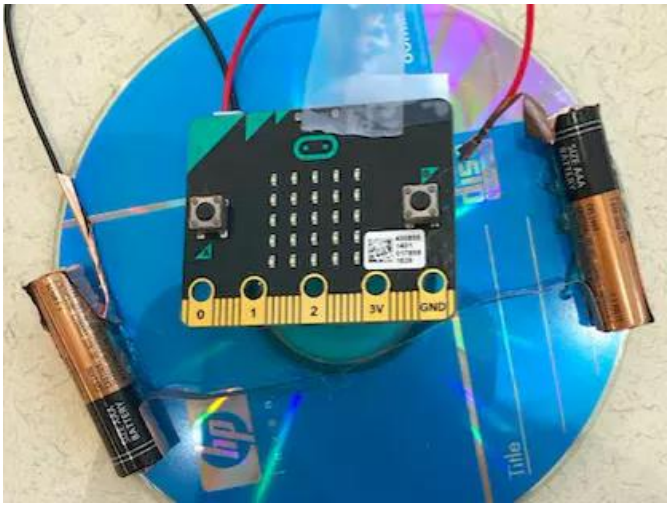
Save, download, and try the program again to make sure that it runs correctly and displays the correct numbers for each variable.

Lesson C: Everything counts

Activity: Everything counts project

This is an assignment for you to come up with a micro:bit program that counts something. Your program should keep track of **input** by storing values in variables and provide **output** in some visual and useful way. You should also perform mathematical operations on the variables to give useful output.

Example: Homemade top with micro:bit revolution counter



Input

Below is a reminder of all the different inputs available to you through the micro:bit:

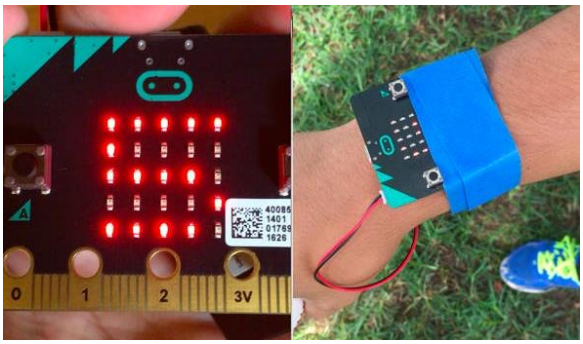
- Acceleration
- Light level
- Button is pressed
- Compass heading
- Temperature
- Running time
- On shake
- On button pressed
- On logo down
- On logo up
- One pin pressed
- On screen down
- On screen up
- Pin is pressed

Project ideas

1. Duct tape wallet

You can see the instructions for creating a durable, fashionable wallet or purse out of duct tape: Duct tape wallet. Create a place for the micro:bit to fit securely. The code at the link above just displays a simple animation. Modify that program, or create a new one that allows you to use Button A to add dollars to the wallet and Button B to subtract dollars from the wallet.
Extra mod: Use other inputs to handle cents and provide a way to display how much money is in the wallet in dollars and cents.

2. Umpire's baseball counter (pitches and strikes)



In baseball, during an at-bat, umpires must keep track of how many pitches have been thrown to each batter. Use Button A to record the number of balls (up to four) and the number of strikes (up to three).
Extra mod: Create a way to reset both variables to zero, create a way to see the number of balls and strikes on the screen at the same time.

3. Shake counter

Using the 'on shake' block, you can detect when the micro:bit has been shaken and increment a variable accordingly. Try attaching the micro:bit to a lacrosse stick and keep track of how many times you have successfully thrown the ball up in the air and caught it.
Extra mod: Have the micro:bit make a sound of increasing pitch every time you successfully catch the ball.

4. Pedometer

See if you can count your steps while running or doing other physical activities carrying the micro:bit. Where is it best mounted?
Extra Mod: Design a wearable band or holder that can carry the micro:bit securely so it doesn't slip out during exercise.

5. Calculator

Create an adding machine. Use Button A to increment the first number, and Button B to increment the second number. Then, use Shake or Buttons A + B to add the two numbers and display their sum.
Extra mod: Find a way to select and perform other math operations.

Reminders about design thinking

Understand the problem

In any design project, it's important to start by understanding the problem. You can begin this activity by interviewing people around you who might have encountered the problem you are trying to solve. For example, if you are designing a wallet, ask your friends how they store their money, credit cards, identification, etc. What are some challenges with their current system? What do they like about it? What else do they use their wallets for?

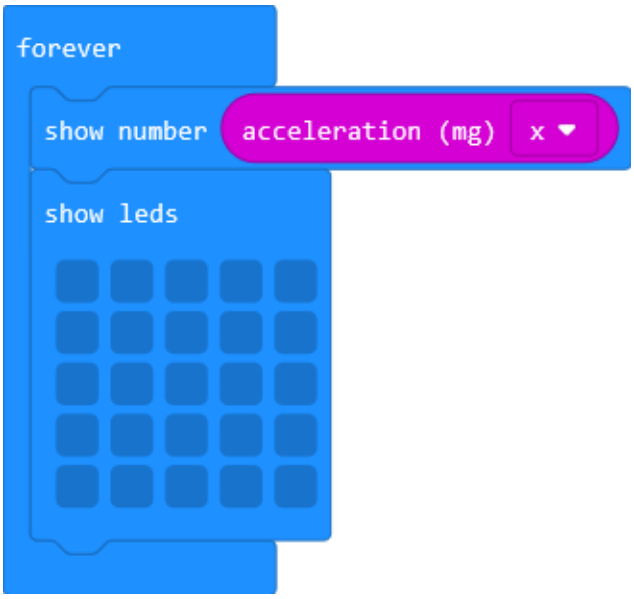
If you are designing something else, think about how you might find out more information about your problem through interviewing or observing people using current solutions.

Prototype

Then, start brainstorming. Sketch out a variety of different ideas. Remember that it's okay if the ideas seem far-out or impractical. Some of the best products come out of seemingly crazy ideas that can ultimately be worked into the design of something useful. What kind of holder can you design to hold the micro:bit securely? How will it be used in the real world as part of a physical design?

Code the program

Use the simulator to do your programming and test out a number of different ideas. What is the easiest way to keep track of data? If you are designing for the accelerometer, try to see what different values are generated through different actions (you can display the value the accelerometer is currently reading using the 'show number' block; clear the screen afterward so you can see the reading).



Project expectations

Follow the design thinking approach and make sure your project meets these specifications:

- Uses at least three different variables in a way that's integral to the program
- Uses unique variable names that clearly describe what information values the variables hold
- Uses mathematical operations to add, subtract, multiply, and/or divide at least two variables
- The program compiles and runs as intended and includes meaningful comments in code
- Provide the written Reflection Diary entry (which we'll talk about after you complete your project)

The design thinking process:

1. **Empathize** by learning more about your target audience.
2. **Define:** understand and identify your audience's problems or needs.
3. **Ideate:** Brainstorm several possible creative solutions.
4. **Prototype:** Construct rough drafts or sketches of your ideas.
5. **Test** your prototype solutions and refine until you come up with the final version.

Use this space to start your design thinking process:

Project scoring rubric

Assessment elements	1	2	3	4
Variables	No variables are implemented.	At least one variable is implemented in a meaningful way.	At least two variables are implemented in a meaningful way.	At least three different variables are implemented in a meaningful way.
Variable names	None of the variable names clearly describe what information values the variables hold.	A minority of variable names are unique and clearly describe what information values the variables hold.	The majority of variable names are unique and clearly describe what information values the variables hold.	All variable names are unique and clearly describe what information values the variables hold.
Mathematical operations	No mathematical operations are used.	Uses a mathematical operation incorrectly or not in a way that is integral to the program.	Uses a mathematical operation on at least one variable in a way that is integral to the program.	Uses a mathematical operation on at least two variables in a way that is integral to the program.
micro:bit program	micro:bit program lacks three or more of the required elements.	micro:bit program lacks two of the required elements.	micro:bit program lacks one of the required elements.	micro:bit program: ▪ Uses variables in a way that is integral to the program ▪ Uses mathematical operations to add, subtract, multiply, and/or divide variables ▪ Compiles and runs as intended ▪ Uses meaningful comments in code

Reflection Diary

Expectations

Write a reflection of about 150–300 words addressing the following points:

- What was the problem you were trying to solve with this project?
- What were the variables that you used to keep track of information?
- What mathematical operations did you perform on your variables? What information did you provide?
- Describe what the physical component of your micro:bit project was (e.g., an armband, a wallet, a holder, etc.)
- How well did your prototype work? What were you happy with? What would you change?
- What was something that was surprising to you about the process of creating this project?
- Describe a difficult point in the process of designing this project and explain how you resolved it.
- Publish your MakeCode program and include the link.

Diary entry scoring rubric

Assessment elements	1	2	3	4
Diary entry	Diary entry is missing three or more of the required elements.	Diary entry is missing two of the required elements.	Diary entry is missing one of the required elements.	Diary entry addresses all elements, including: ▪ Brainstorming ideas ▪ Construction ▪ Programming ▪ Beta testing

Glossary of key terms

Constants: Computer input that is stored and used in a computer program with a value that does not change throughout the course of the program.

Variables: A container for data. Every variable has a name that is used to reference the data that it contains. Every variable also has a variable type.

Variable types: The type of data that a variable can contain:

- **Arrays:** A type of variable that holds a list of multiple items.
- **Booleans:** A variable type that can be either true or false. A Boolean condition is a condition that evaluates to either true or false.
- **Number:** A variable type that holds numerical data.
- **Sprites:** A variable that represents a single dot on the screen and holds two separate values for the row and column the dot is currently in.
- **String:** A variable type that holds sequence of alphanumeric characters and/or symbols.